AutoVet Testing Summary

Unit, Integration, System, Security & Performance, User Acceptance, and Automated Testing: what AutoVet has, explained in simple terms.
Test run: 31 October 2026 · 66 tests passed (403 checks) in about 56 seconds

At a glance

Testing type	Status	What exists
1. Unit Testing	Done	8 tests, all passed (AI forecast maths)
2. Integration Testing	Done	58 tests on a real MySQL database, all passed
3. System Testing	Partly done	Full system deployed; the manual test record still needs to be written
4. Security Testing	Done	Security features built in and checked by tests
4. Performance Testing	Partly done	Speed features built in; no load test and no speed measurements yet
5. User Acceptance Testing	Ready	Three ISO 9241-11 questionnaires are prepared; results still need to be collected
6. Automated Testing	Done	GitHub Actions runs every test and check on each code change

> **Important.** Only write in your paper what was actually done. Wherever this document marks a **Gap**, either do that test before submitting or describe it as a limitation and recommendation. Do not invent numbers.

Tools used

Tool	What it is used for
PHPUnit (through Laravel)	Writing and running the unit and integration tests (`php artisan test`).
MySQL 8.4 test database	A separate, throwaway database (`autovet_test`) so tests never touch real data.
GitHub Actions	Runs all the tests and checks automatically whenever code is pushed.
npm audit	Checks the website packages for known security vulnerabilities.
Printed / online questionnaires	User Acceptance Testing with clinic staff and pet owners.

1. Unit Testing

In simple terms: unit testing checks **one small piece of code on its own**, without the database or the website. It is like testing one part of a car engine on a table before putting the car together.

What is tested in AutoVet

The unit tests are in `backend/tests/Unit` and focus on the maths behind the **AI inventory forecast**:

Test	What it proves
Reproduces the worked example	The best-fit line, daily usage rate (slope), R^2, RMSE and MAE match an example calculated by hand.
Test days projected from the last training total	The 80/20 test predicts from the last known total, the same way the system predicts stock-outs.
Benchmark methods project from the last training total	The average, naïve and 7-day moving average methods give the correct numbers.
Benchmark methods count days without usage as zero	Days when nothing was used are handled correctly.
Fewer than ten points are scored in-sample	Items with little history are handled safely.
Confidence labels match the forecast	R^2 of 0.70 or more = High, 0.40 or more = Medium, lower = Low.
Random draws average out to the expected rate	The demo-data generator (Poisson) produces realistic usage.
Basic example test	Laravel's default check that the test setup works.

> **Example.** A syringe was used 2, 3, 2, 3, 3, 4, 3, 5 times over 8 days. Worked by hand, the daily rate is 3.20. The unit test feeds the same numbers to the code and checks that it also gets 3.20. If someone later changes the code by mistake, this test fails right away.

Result: 8 unit tests, **all passed**.

SAMPLE PARAGRAPH

Unit testing was performed with PHPUnit on the forecasting component of AutoVet. Eight unit tests verified the linear regression calculations (slope, R^2, RMSE and MAE), the 80/20 holdout procedure, the three baseline forecasting methods, the confidence labels, and the demo-data generator. All eight tests passed.

2. Integration Testing

In simple terms: integration testing checks that **the parts work together**. A test sends a real request to the system, as if a user clicked a button. The request passes through login, permissions and the program logic, and is saved in a real database. The test then checks the result.

What is tested in AutoVet

The integration tests are the 58 **feature tests** in `backend/tests/Feature`. They run against a real MySQL database.

Area	Tests	Examples of what is checked
Access control	9	A vet cannot create a clinic admin or promote themselves; an admin cannot create a super admin; a vet cannot reset an admin's password.
Login and account security	11	A wrong email and a wrong password show the same message; inactive, suspended or archived accounts are blocked; a password reset logs out other sessions.
Clinic separation	4	Staff cannot use another clinic's pet; each clinic has its own notification channels.
Client portal limits	5	Client bookings always start as "pending"; clients cannot approve their own appointment, open draft invoices or write medical records.
Appointments	3	Saving an appointment with a status, blocking double-booking of the same pet, and updating the status.
Invoices	3	Invoice numbers continue from the highest one; a finalized invoice cannot go back to draft.
Backups	2	Backups are saved in the clinic's own folder; one clinic cannot download another clinic's backup.
Data exposure	5	Clients only see what they are allowed to; the status page reveals nothing about the database; security headers are present.
Lists and search	5	Page-size limits, allowed sort columns, safe search, and validated report filters.
Dashboard	3	Admins get every section; staff do not see sales; one failing section does not break the rest.
Error handling	2	Users see friendly messages; internal error details stay hidden.
Demo data tools	5	Demo data is labelled, never overwrites real records, and can be fully removed.
Basic check	1	The application starts and responds.
Total	**58**	**All passed**

> **Example.** The double-booking test books a pet for an appointment, then tries to book the same pet again at the same time. The system must refuse the second booking. The test passes only if the second booking is rejected and nothing extra is saved in the database.

SAMPLE PARAGRAPH

Integration testing was carried out with 58 Laravel feature tests that send real HTTP requests to the API and check the results in a MySQL database. The tests covered role-based access control, authentication security, separation between clinics, client portal restrictions, appointments, invoicing, backups, data exposure, list and search validation, the dashboard, error handling and the demo data tools. All 58 tests passed.

3. System Testing

In simple terms: system testing checks **the whole finished system** (admin web app, client portal, backend and database together) to see if it does everything promised in the scope.

What AutoVet has

- The full system is deployed: the frontend on **Vercel**, and the backend on **Render** with a **TiDB (MySQL)** database and **Supabase** file storage.
- The integration tests in Section 2 already check many features at the server level.

> **Gap.** There are no automated tests that click through the screens, and there is no written system test record yet. Before submitting, run each module by hand on the live system and fill in a table like the one below.

Template for your system test record

Test ID	Module	Steps	Expected result	Actual result	Pass / Fail
ST-01	Login	Log in as a vet with the correct password	Vet dashboard opens		
ST-02	Login	Log in with a wrong password	"Invalid credentials..." message is shown; user is not logged in		
ST-03	Appointments	Client books an appointment in the portal	Booking appears as "Pending" for staff		
ST-04	Billing	Create an invoice with a service and a product	Correct subtotal, discount and total		
ST-05	Inventory / AI	Open the forecast for an item with usage history	Days until stock-out and confidence level shown		

Modules to cover (from the scope): login and roles, appointments, medical records, billing and invoices, inventory and the AI forecast, reports, client portal, notifications, backups, and audit logs.

The five rows above are examples only. Fill in "Actual result" and "Pass / Fail" from what really happens when you run them.

4. Security & Performance Testing

Security (implemented and tested)

In simple terms: security testing checks that the system keeps out people who should not get in, and protects data from being seen or changed by the wrong users.

Protection	How AutoVet does it
Login protection	Laravel Sanctum tokens. A wrong email and a wrong password show the same message, so attackers cannot discover which accounts exist. Inactive, suspended and archived accounts are blocked. A password reset ends all other sessions.
Rate limiting (stops password-guessing)	Login: 10 tries per minute. Register, forgot password and reset password: 5 per minute. Google sign-in: 10 per minute. AI clinical support: 10 per minute. Client email sending: 30 per minute. General API: 300 per minute per user.
Role-based access	Each role (super admin, clinic admin, veterinarian, staff, client) can only do what it is allowed to. Checked by the access-control and portal tests.
Clinic separation	One clinic can never see another clinic's pets, notifications or backups. Checked by the clinic-separation and backup tests.
Security headers	HSTS, Content-Security-Policy, X-Frame-Options (DENY), X-Content-Type-Options (nosniff), Referrer-Policy and Permissions-Policy are sent, and the X-Powered-By header is removed. Checked by a test.
Safe input handling	Only allowed sort columns are accepted, search wildcards are treated as plain text, page sizes have a maximum, and report filters are validated.
Package vulnerability check	On every code push, `npm audit` checks both websites and fails if a high or critical vulnerability is found.

> **Example.** Someone tries to guess a password and submits the login form 11 times in one minute. On the 11th try, AutoVet blocks the request and asks them to wait, so passwords cannot be guessed quickly.

Performance (built in, not yet measured)

In simple terms: performance testing checks that the system stays **fast**, even when many people use it.

- **Database indexes** make searches and lookups faster. Several migrations add indexes for search fields and linked records.
- **Caching** keeps frequently used results ready instead of recalculating them (used in 9 places).
- **A page-size limit** stops anyone from loading huge lists at once.
- **The dashboard** loads all its sections in one request instead of many.

> **Gap.** No load or stress test has been run, so there are **no response-time numbers** yet. Do not claim speeds you have not measured. Choose one:
>
> 1. **Measure it.** Open the main pages (login, dashboard, appointment list, inventory forecast) on the live system 5 times each, note the load time from the browser's developer tools (Network tab), and report the average.
> 2. **Recommend it.** State in Chapter 5 that load testing with many users at the same time is recommended for future work.

SAMPLE PARAGRAPH (SECURITY)

Security was addressed through token-based authentication with Laravel Sanctum, rate limiting on login and account recovery, role-based access control, separation of data between clinics, HTTP security headers, and input validation. These protections were verified by the automated feature tests, and the frontend packages were checked for known vulnerabilities with npm audit on every code change.

5. User Acceptance Testing (UAT)

In simple terms: UAT asks the **real users** whether the system works well for them and whether they would accept it for daily use.

The UAT instruments

AutoVet has three questionnaires for **Pet Wellness Animal Clinic**, based on **ISO 9241-11:2018**. That standard defines usability as how well users reach their goals with **effectiveness**, **efficiency** and **satisfaction**.

Respondent group	A. Effectiveness	B. Efficiency	C. Satisfaction	D. Admin controls	Items
Veterinarians and administrators	1–12	13–23	24–34	35–39 (admins only)	39
Front desk staff	1–10	11–20	21–30	—	30
Clients / pet owners	1–10	11–18	19–27	—	27

Each questionnaire has three sections: **Section 1** asks for general and demographic information, **Section 2** evaluates the system using a 4-point scale, and **Section 3** asks for open-ended feedback.

1	2	3	4
Strongly Disagree	Disagree	Agree	Strongly Agree

> **Gap.** The questionnaires are ready, but no results are recorded yet. After collecting them, report the following in Chapter 4.

How to report the results

1. The number of respondents in each group.
2. The **mean (average) score** for each section: Effectiveness, Efficiency, Satisfaction, and Admin controls for administrators.
3. A verbal interpretation of each mean. A common scale for a 4-point survey is shown below. Use the scale written in your Chapter 3 if it differs.

Mean score	Interpretation
3.26 – 4.00	Strongly Agree
2.51 – 3.25	Agree
1.76 – 2.50	Disagree
1.00 – 1.75	Strongly Disagree

> **Example of computing a mean (made-up numbers).** Five front desk staff rate the statement "Booking an appointment is quick" as 4, 3, 4, 4 and 3. The mean is (4 + 3 + 4 + 4 + 3) ÷ 5 = **3.60**, which falls in 3.26–4.00, so the interpretation is "Strongly Agree".

6. Automated Testing

In simple terms: automated testing means the computer runs the tests by itself, every time the code changes, without anyone pressing a button. If something breaks, the team knows right away.

How AutoVet does it

AutoVet uses **GitHub Actions** (`.github/workflows/ci.yml`). It runs on **every push and pull request** to the `Seb1` and `main` branches, in three parallel jobs:

Job	Steps
Backend (PHP 8.2)	1. Check that `composer.json` is valid. 2. Install packages. 3. Check every PHP file for syntax errors. 4. Run all **66 PHPUnit tests** against a real **MySQL 8.4** database.
Client portal	1. Install packages. 2. Run the security audit (`npm audit` , fails on high or critical issues). 3. Build the website.
Admin app	Same steps as the client portal.

If any step fails, the whole run is marked **failed (red)**, so broken code is caught before it is used.

> **Example.** A developer accidentally changes the invoice code so that a finalized invoice can be turned back into a draft. When they push the change, GitHub Actions runs the tests, the invoice test fails, and the run turns red before the mistake reaches the clinic.

> **Note.** The React frontends have **no automated tests of their own**. The automation only checks that they build successfully and that their packages have no serious vulnerabilities. Say this plainly if your paper describes frontend testing.

SAMPLE PARAGRAPH

Automated testing was set up with GitHub Actions. On every push and pull request, the pipeline validates the backend configuration, checks all PHP files for syntax errors, runs the full suite of 66 PHPUnit tests against a MySQL 8.4 database, audits the frontend packages for high-severity vulnerabilities, and builds both frontend applications. Any failure stops the pipeline, so errors are detected before deployment.

Overall summary

SAMPLE SUMMARY PARAGRAPH

AutoVet was tested at several levels. Eight unit tests verified the AI forecasting calculations, and 58 integration tests verified the system's features, access rules and security on a real database. All 66 tests (403 checks) passed and run automatically on every code change through GitHub Actions. Security protections are implemented and tested. Performance features such as indexing, caching and page limits are in place, and user acceptance was evaluated with ISO 9241-11-based questionnaires for veterinarians and administrators, front desk staff, and pet owners.

Before you use the summary paragraph, add your UAT results, and your speed measurements if you take them. If you skip performance measurement, change that sentence so it does not imply speeds were measured. Test counts are from the run on 31 October 2026; if tests are added later, re-run `php artisan test` and update them.